

ECE27I

Final Review

Z. Gu

Fall 2025

Character String

`char str[13] = "ARM Assembly";`

Memory Address	Memory Content	Letter
str + 12 →	0x00	\0
str + 11 →	0x79	y
str + 10 →	0x6C	l
str + 9 →	0x62	b
str + 8 →	0x6D	m
str + 7 →	0x65	e
str + 6 →	0x73	s
str + 5 →	0x73	s
str + 4 →	0x41	A
str + 3 →	0x20	space
str + 2 →	0x4D	M
str + 1 →	0x52	R
str →	0x41	A

Data Representation

- ▶ Integers
 - ▶ Binary in different bases: binary, octal, hex, decimal
 - ▶ Signed Integers
 - ▶ Signed magnitude
 - ▶ One's complement
 - ▶ Two's complement
 - ▶ Arithmetic Operations
 - ▶ N, V, C, Z flags
 - ▶ Big Endian vs Little Endian
- ▶ ASCII values
 - ▶ Null-terminated string
 - ▶ Converting between numbers and ASCII
 - ▶ Upper case, lower case
- ▶ Fixed pointer numbers
 - ▶ Qm.n format
 - ▶ Addition, subtract, multiplication, division

Basic Assembly Programming

- ▶ Load-modify-store sequence
- ▶ Accessing memory
 - ▶ Memory addressing mode
 - ▶ Pre-index
 - ▶ Post-index
 - ▶ Pre-index with update
- ▶ Data processing
 - ▶ Arithmetic, Logic, Comparison, Data Movement
 - ▶ Barrel Shifter:
 - ▶ `ADD r1, r0, r0, LSL 2`
 - ▶ Bit operations
 - ▶ Set a bit, Reset a bit, Toggle a bit, Check a bit
 - ▶ `LSL, LSR, ASR, ROR, RRX`
- ▶ Flow control: if, if-then-else, for loop, while loop
 - ▶ Unconditional Branch: `B`
 - ▶ Conditional Branch: `CMP, TEQ, TST, BEQ, BNE, BMI, BLS, BHI`, etc.
 - ▶ Conditional Execution: `MOVEQ, MOVNE`

Subroutine Without Floating Numbers

- ▶ Calling Subroutine & Exiting Subroutine
 - ▶ BL and BX
 - ▶ How PC, LR, and SP registers are updated?
- ▶ Embedded application binary interface (EABI) Protocol
 - ▶ Save and restore R4-R11, LR if your subroutine wishes to modify
 - ▶ Parameters passing in registers R0, R1, R2, R3
 - ▶ Extra parameters passing in stack
 - ▶ Return parameter in R0
- ▶ Recursive functions
 - ▶ How does the stack work?
 - ▶ Pay special attention to LR register

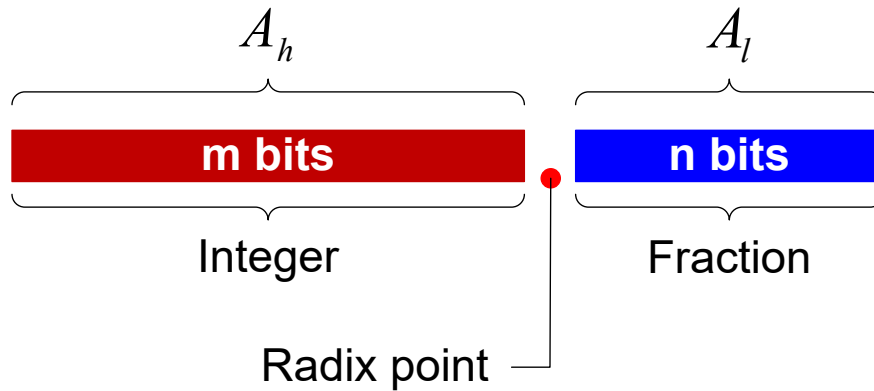
Subroutine with Float Arguments

- ▶ Each number is assigned in turn to the next free register of the corresponding type
- ▶ Example:
 - ▶ `double fun(double a1, float a2, double a3, float a4, float a5, double a6, float a7, double a8)`

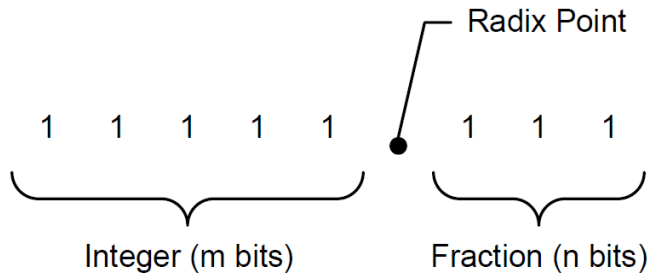
Double-Precision View	D0		D1		D2		D3		D4		D5		D6		D7	
Single-precision View	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15
Argument View	a1		a2	a4	a3		a5	a7	a6		a8					

Unsigned Fixed-point Representation

UQm.n



$$f = A_h + A_l \times 2^{-n}$$



$$\begin{aligned} 10101.101_2 &= A_h + A_l \times 2^{-3} \\ &= 21 + 5 \times 2^{-3} \\ &= 21.625 \end{aligned}$$

Signed Fixed-point Representation

Qm.n

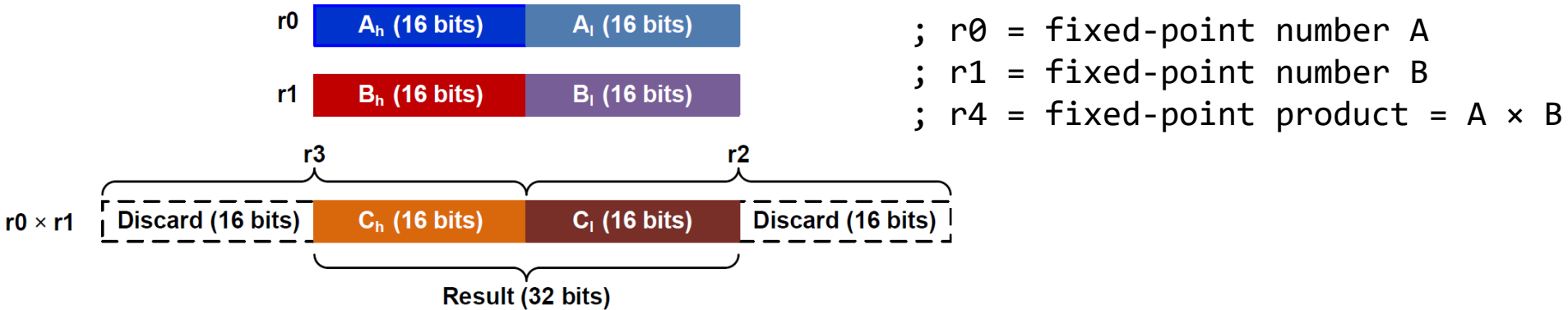


$$A = -1 \times b_{N-1} \times 2^{N-1} + \sum_{i=0}^{N-2} (b_i \times 2^i)$$

$$f = \frac{A}{2^n}$$

where $N = m + n + 1$

Fixed-point Multiplication in **Q15.16**



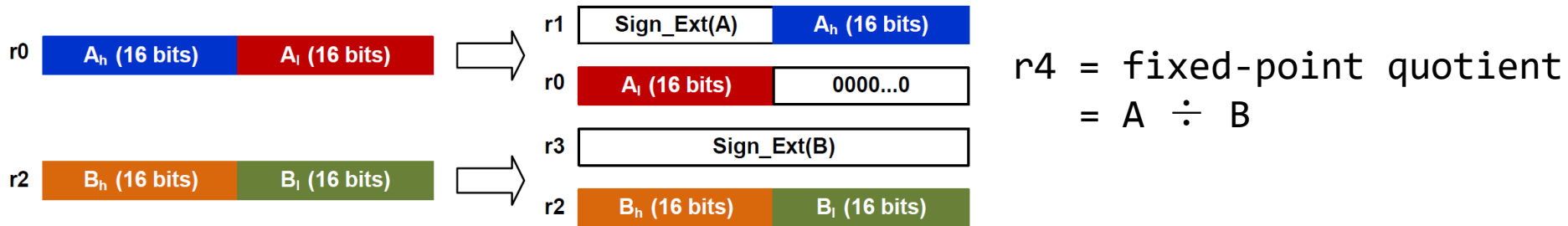
```
SMULL r2, r3, r0, r1    ; Unsigned long multiply
                        ; r2 = low word,  r3 = high word
```

```
LSLS    r3, r3, #16        ; Shift left high word
```

```
LSRS    r2, r2, #16        ; Shift right low word
```

ORR r4, r2, r3 ; Pack two halfwords into a word

Fixed-point Division in UQ16.16 or Q15.16

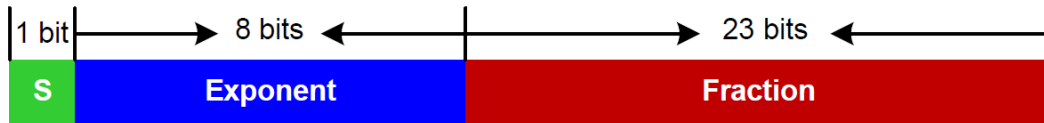


```

ASRS  r1, r0, #31          ; r1[31:0] = sign bits of A
LSLS  r1, r1, #16          ; put sign bits in upper half word
ORR   r1, r1, r0, LSR #16
LSLS  r0, r0, #16          ; [r1:r0] stores sign extended A
ASRS  r3, r2, #31          ; [r4:r3] stores sign extended B
BL    division_64_bits    ; Take four input register (r0:r1, r3:r4)
MOV   r4, r0               ; division_64_bits places result in r0
    
```

IEEE Std 754

Single Precision (32 bits)



Double Precision (64 bits)



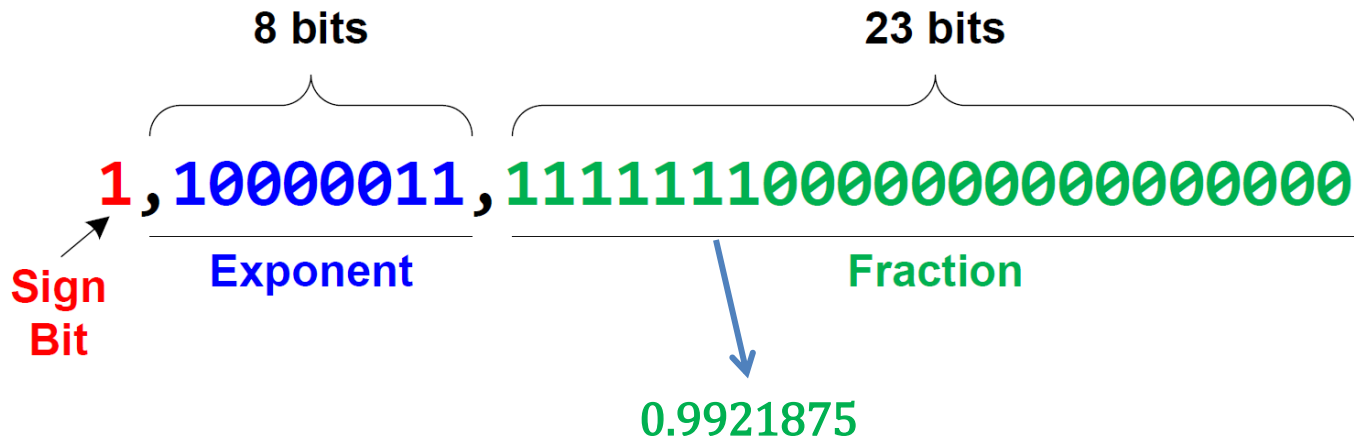
IEEE 754 value:

$$(-1)^S \times (1 + \textit{Fraction}) \times 2^{\textit{Exponent} - \textit{Bias}}$$

where Bias = **127** for single precision

Bias = **1023** for double precision

Decoding 0xC1FF0000 into a floating-point number



$$\begin{aligned} f &= (-1)^S \times (1 + \textit{Fraction}) \times 2^{\textit{Exponent} - 127} \\ &= (-1)^1 \times (1 + 0.9921875) \times 2^{131 - 127} \\ &= -1 \times 1.9921875 \times 2^4 \\ &= -31.875 \end{aligned}$$

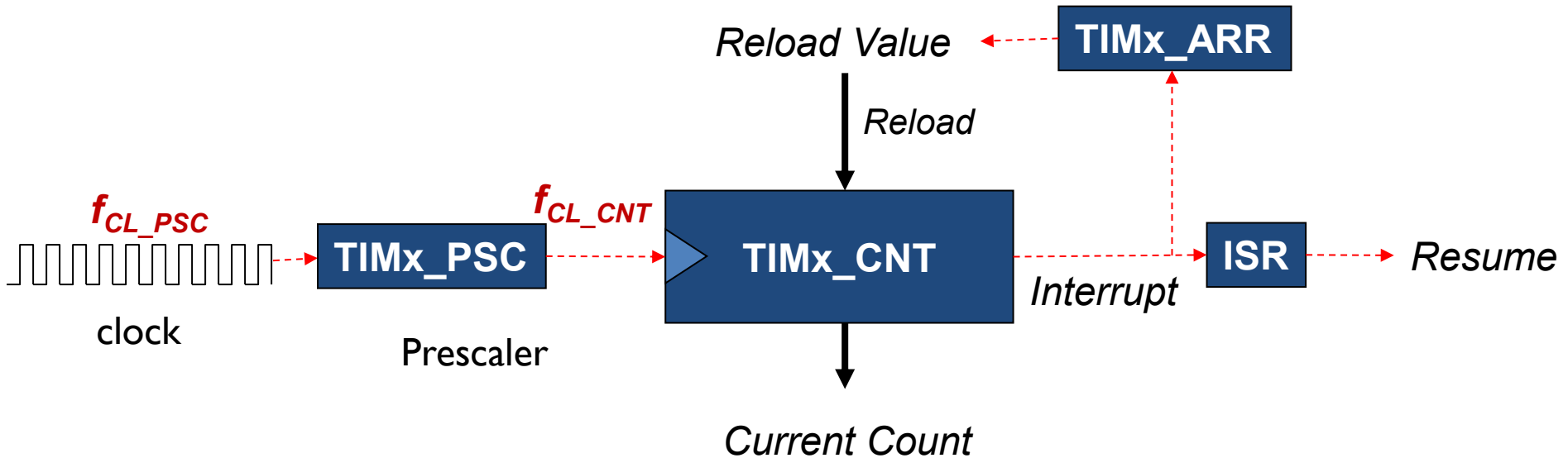
Encoding 14.5 into IEEE Std 754 Single-Precision

Normalization:

$$\begin{aligned} 14.5 &= 1.8125 \times 2^3 \\ &= (1 + 0.8125) \times 2^3 \end{aligned}$$

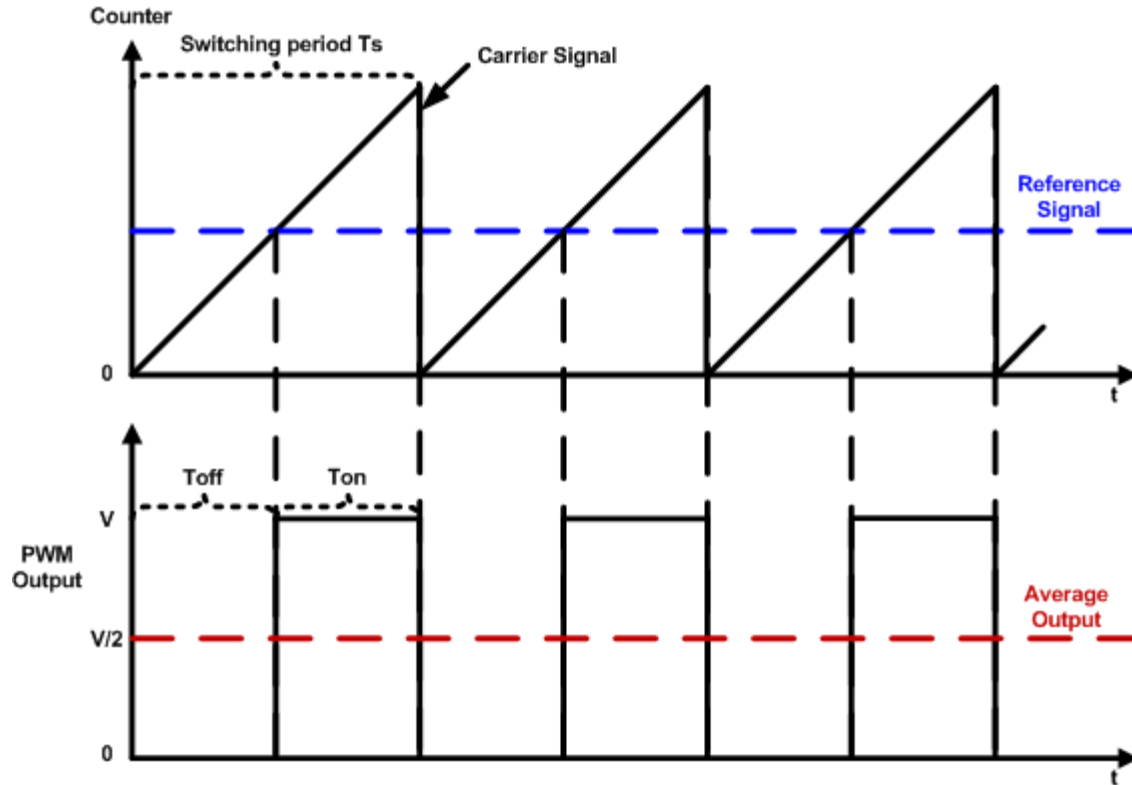
- ▶ *Sign* = 0
- ▶ *Exponent* = $3 + 127 = 130 = 1000010_2$
- ▶ *Fraction* = 0.1101_2
 - ▶ $0.8125 \times 2 = 1.625 = 1 + 0.625$
 - ▶ $0.625 \times 2 = 1.25 = 1 + 0.25$
 - ▶ $0.25 \times 2 = 0.5 = 0 + 0.5$
 - ▶ $0.5 \times 2 = 1$
- ▶ $14.5 = 01000001011010000000000000000000$ in binary or 0x41680000 in hex

Timer's Clock



$$f_{CK_CNT} = \frac{f_{CL_PSC}}{PSC + 1}$$

PWM Duty Cycle = $T_{on}/\text{Time Period}$



$$\text{duty cycle} = \frac{\text{pulse on time}}{\text{pulse switching period}} \times 100\% = \frac{T_{on}}{T_s} \times 100\%$$